Defining the Scope

IN-SCOPE (The Core Build)

- ✅ **Fare Estimation** (Input locations -> Get Price)
  JetBrains Mono
- ✅ **Ride Request** (Book based on estimate)
  JetBrains Mono
- ✅ **Matching** (Real-time driver discovery)
  JetBrains Mono
- ✅ **Driver Workflow** (Accept/Decline & Navigate)
  JetBrains Mono

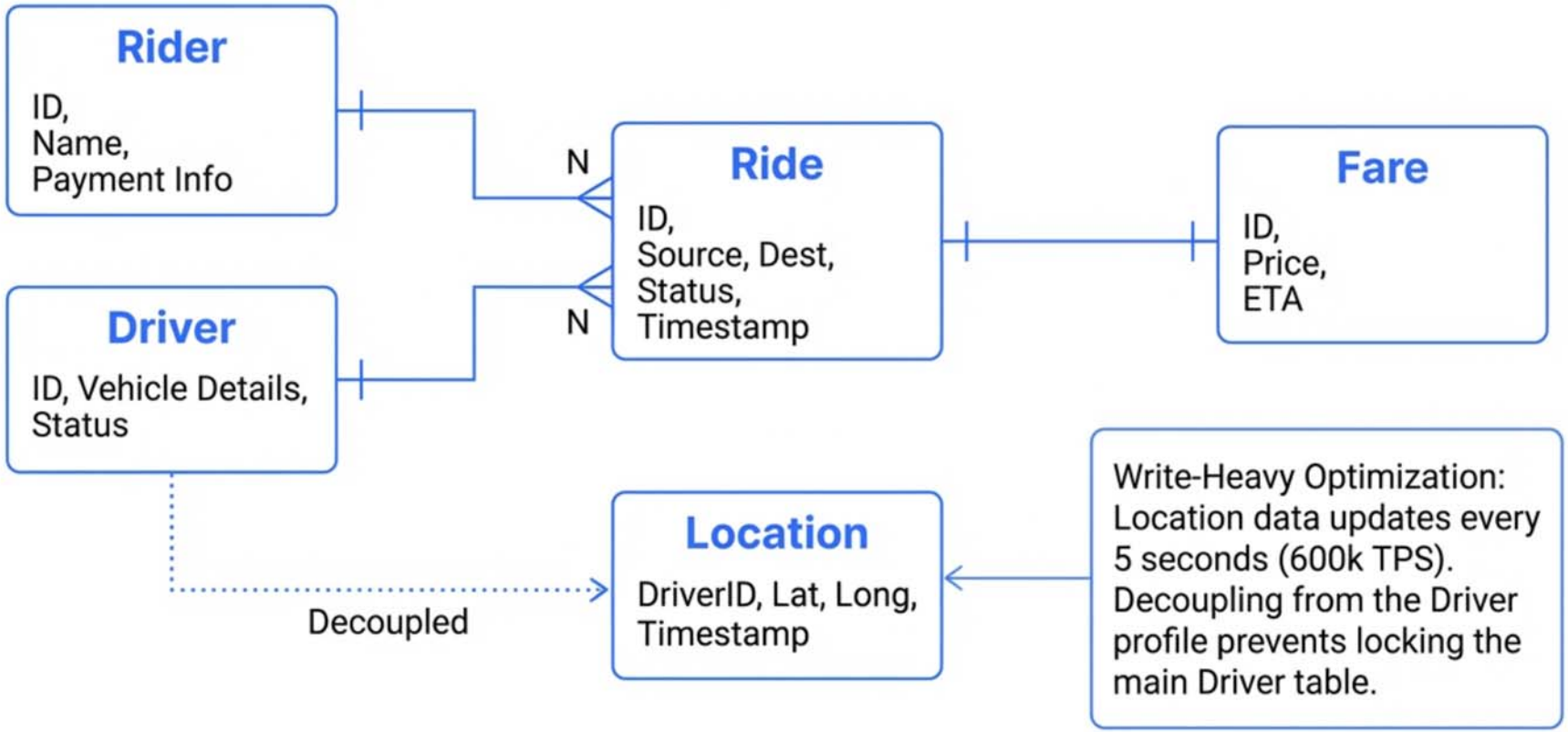
Non-Functional Requirements

- Low Latency Matching (< 1 min)
- Strong Consistency (No double bookings)
- High Availability (Survive regional failures)
- High Throughput (Handle 'Taylor Swift' surges)

OUT-OF-SCOPE (The Red Velvet Rope)

- Ratings & Reviews ✗
- Scheduled Rides ✗
- Ride Types (XL, Pool, Pet) ✗
- Payment Processing Specifics ✗

The Data Foundation: Core Entities



The Interface: API Design

1. Estimate Fare

```
POST /fare
Body: { pickup, destination }
Returns: { fareId, price, eta }
```

2. Request Ride

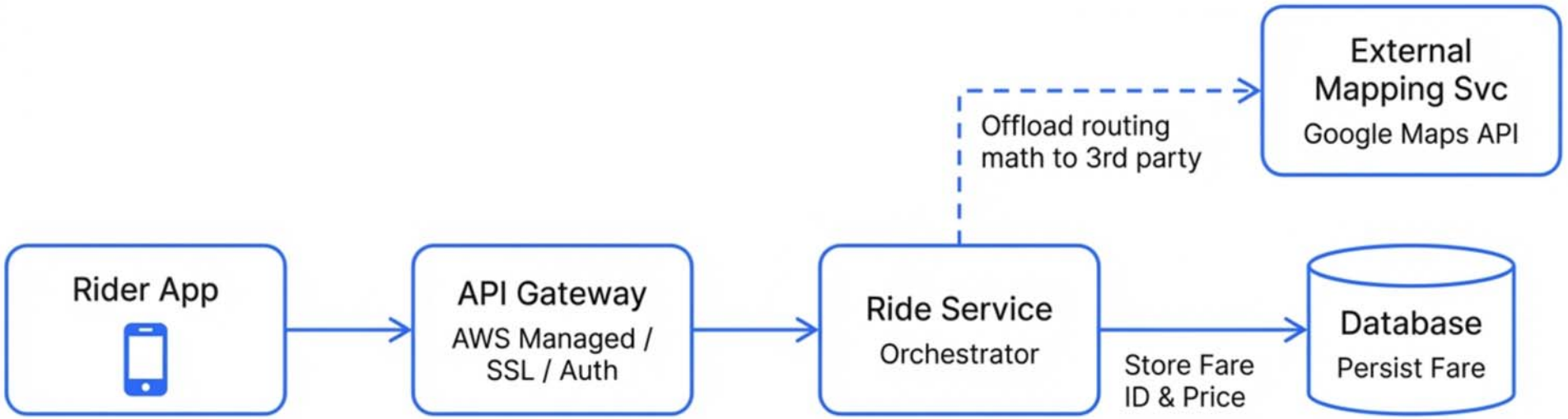
```
POST /rides
Body: { fareId }
Returns: { rideId, status: 'requested' }
```

3. Update Location

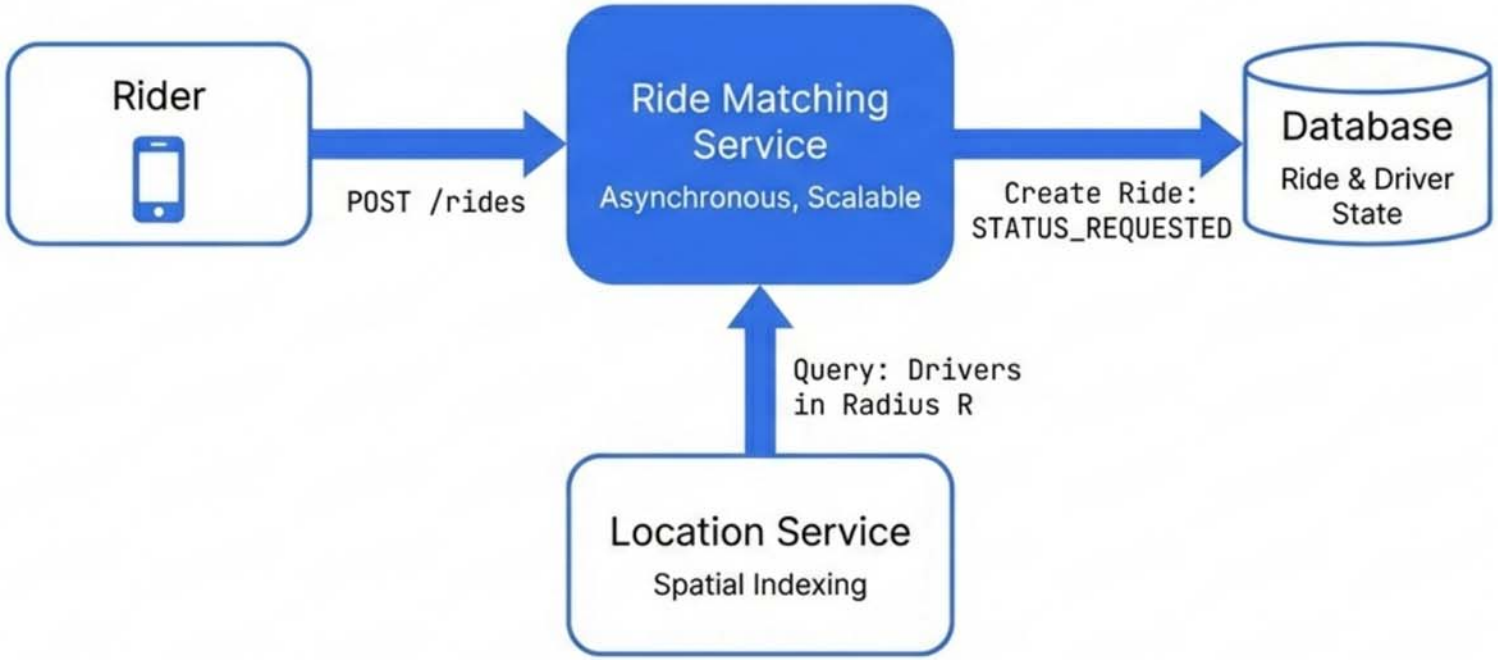
```
POST /drivers/location
Body: { lat, long }
// Note: UserID extracted from JWT
```

Security Note: Security Best Practice: UserIDs are NOT passed in the request body. They are extracted securely from the Session Token/JWT header to prevent IDOR attacks.

Phase 1: The Fare Estimate (Happy Path)



Phase 2: The Matching Logic



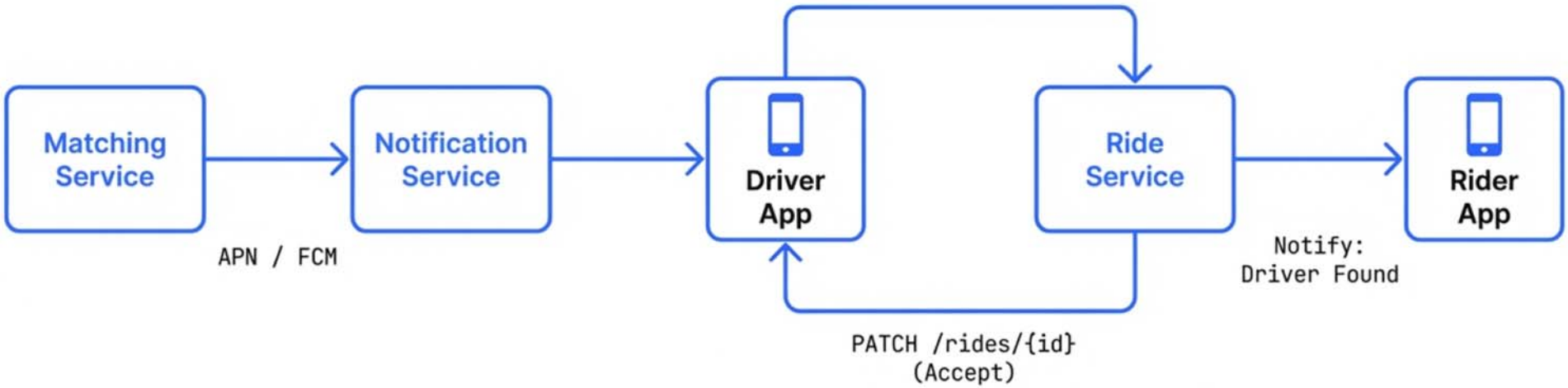
Separation of Concerns

The Matching Service is decoupled from the main Ride Service.

Matching is a heavy, asynchronous process (up to 1 min) and scales independently.

Phase 3: The Driver Loop

The Bidirectional Handshake



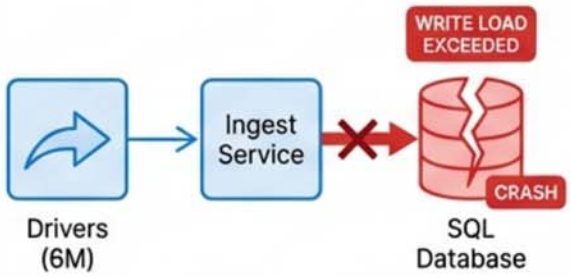
State synchronization is critical. The offer must be locked and accepted in real-time.

The Stress Test: Why the Happy Path Fails

The system works for 100 users. Does it work for 100 Million?

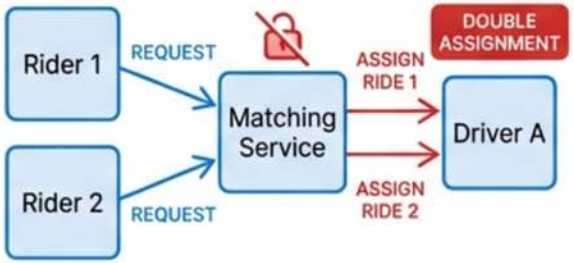
The Ingest Problem

6 Million drivers x 5-second updates = `600,000` TPS. A standard SQL database will crash under this write load



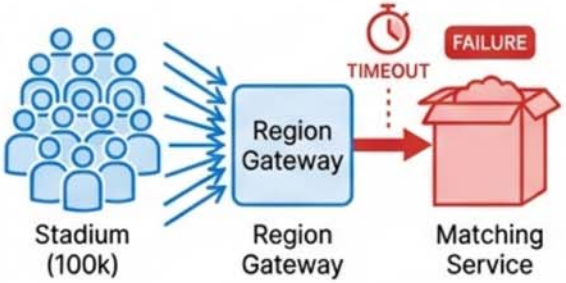
The Race Condition

Two riders request the same driver at the exact same `millisecond`. Without locking, one driver gets two rides.



The Surge

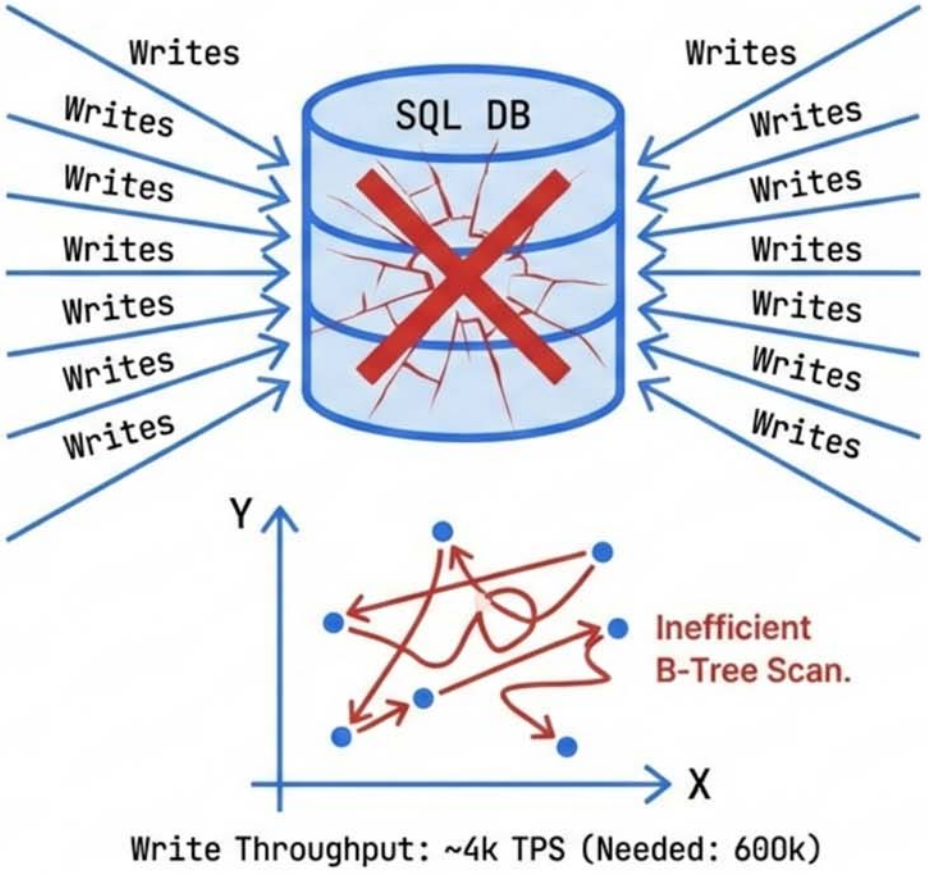
A stadium empties. `100,000` requests hit a single region instantly. Synchronous matching will timeout and fail.



Deep Dive 1: The Location Bottleneck

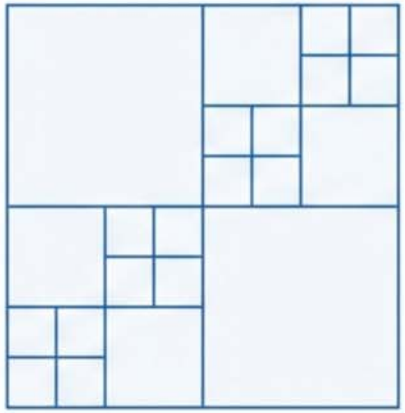
The Strawman: Storing Lat/Long in Postgres.

Why it fails: B-Tree Indexes are 1-Dimensional. They cannot efficiently index 2D geospatial data without massive row scanning. Standard SQL write throughput is ~2-4k TPS. We need 600k.



Solution: In-Memory Geohashing

Quadtree (Yelp style)



Good for static data. Bad for high updates (re-indexing is expensive).

Geohashing (Uber style)

s912 s912	s913 s913	s914 s914
s925 s925	s926 s926	s927 s927
s938 s938	s939 s939	s93a s93a

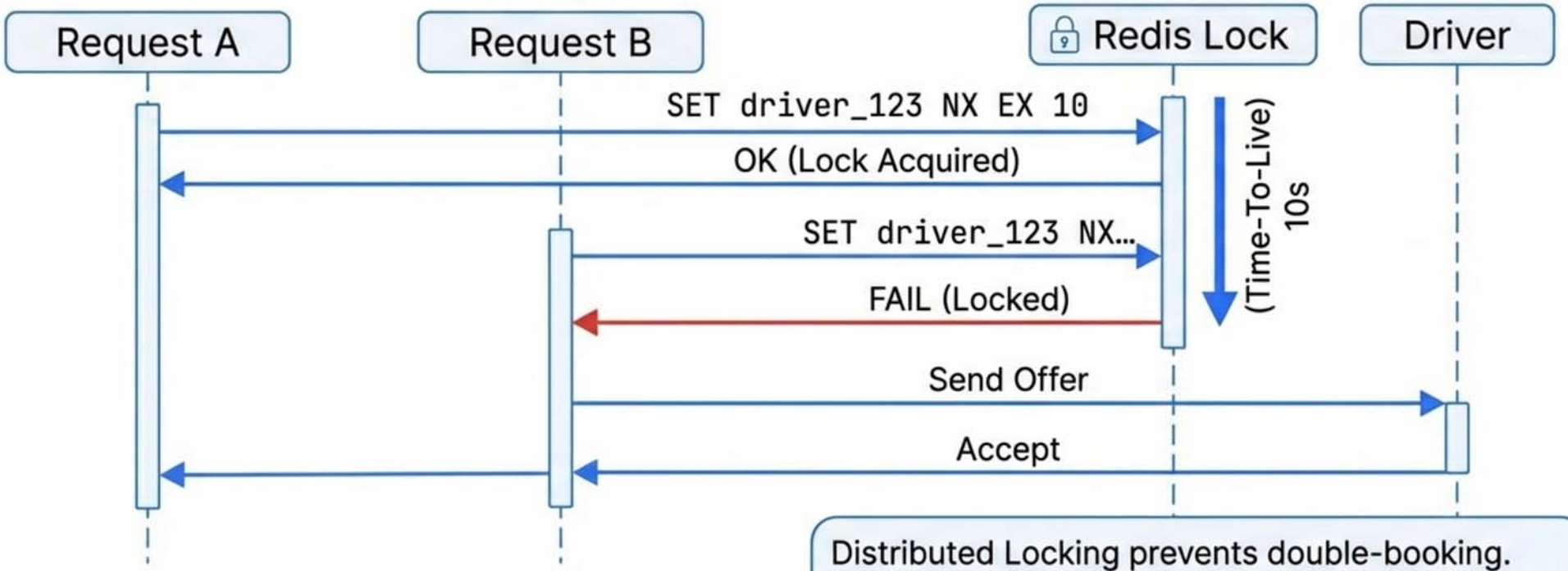
Redis + Geohash. Indiscriminate grid. Fast updates (O(1)).

Winner for Ride-Sharing.

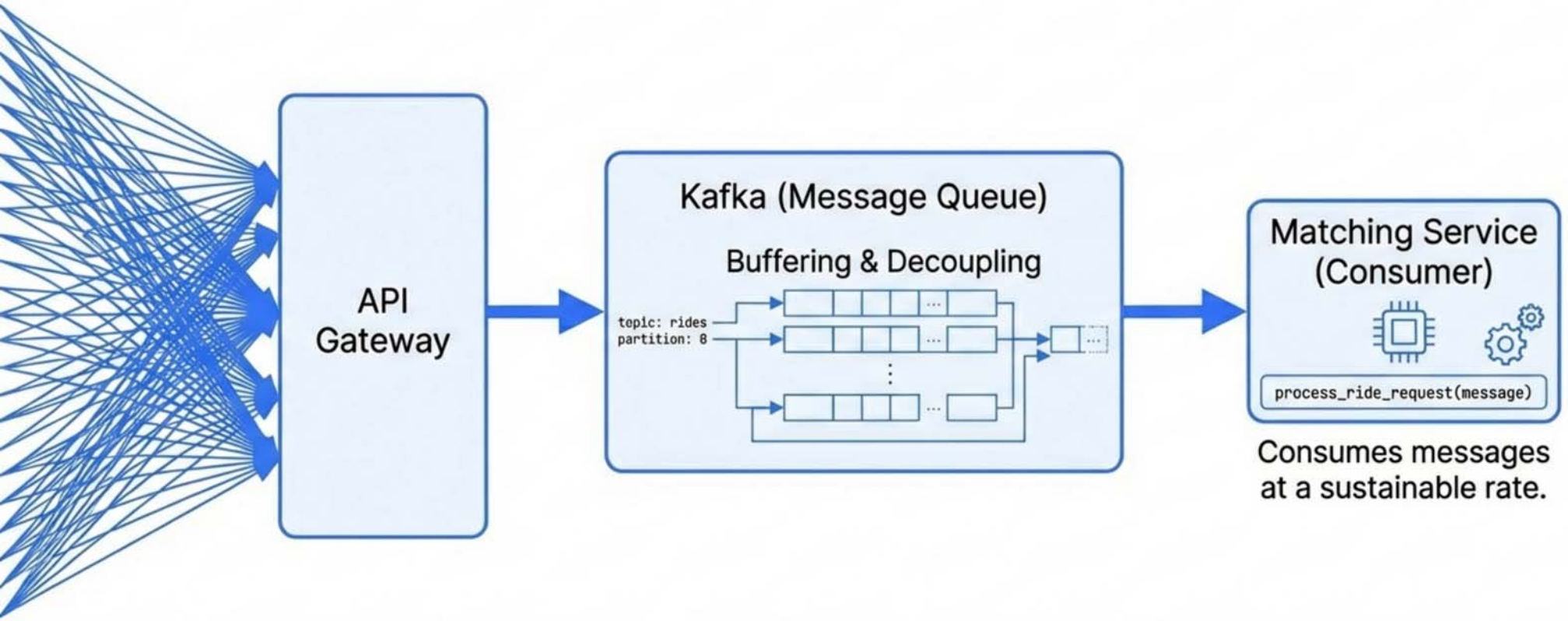
```
Redis commands:
GEOADD, GEOSEARCH
```

Deep Dive 2: Consistency & The Race Condition

The Ticketmaster Analogy



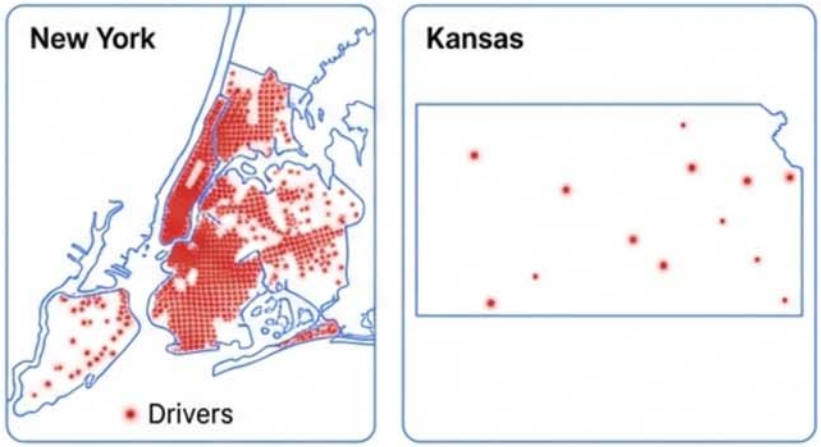
Deep Dive 3: Handling the “Taylor Swift” Surge



Asynchronous processing prevents the Matching Service from crashing under load. The queue absorbs the pressure, allowing the system to process requests without dropping them.

Scaling Strategy: Sharding & Hotspots

The Hotspot Problem

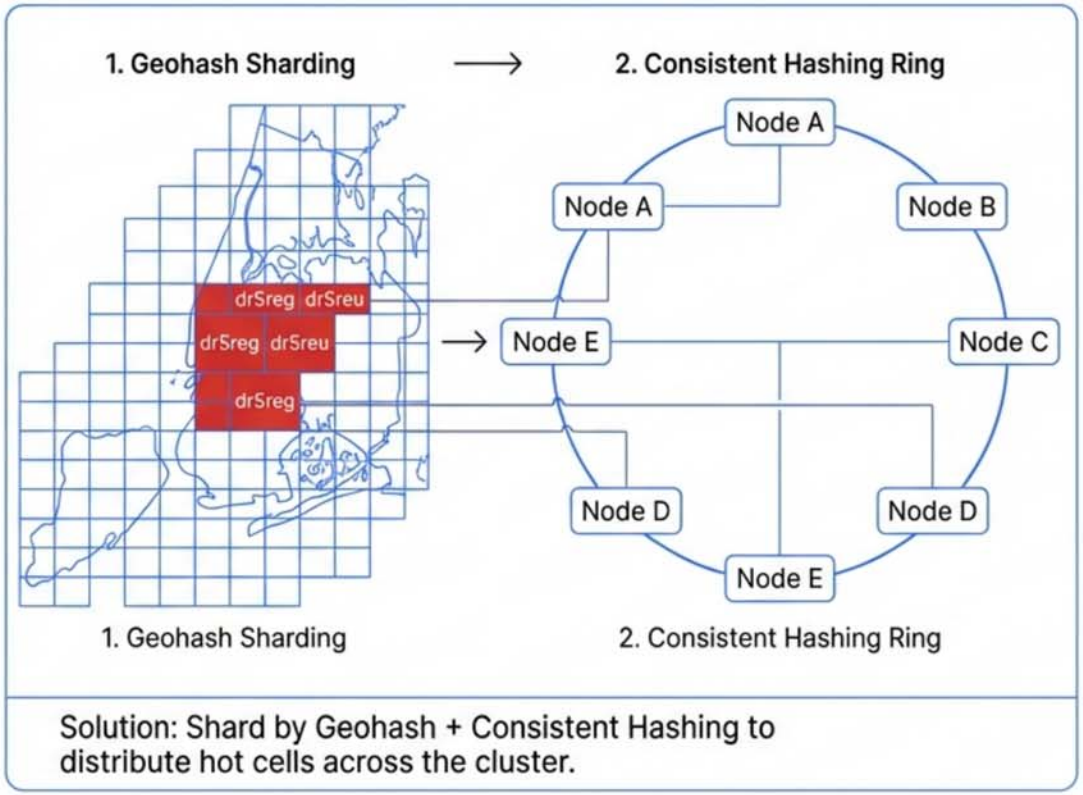


Dense Driver Concentration (Hotspot)

Sparse Driver Concentration

Problem: Sharding by City melts the 'New York' server. Sharding by Driver ID makes proximity search impossible.

Hybrid Sharding Strategy



Solution: Shard by Geohash + Consistent Hashing to distribute hot cells across the cluster.

The Final Blueprint